# Trees

DSA27 — Lecture 11 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 11 · Fall 2026

# Where We Are

# From lines to branches

Every structure so far was **linear**: array, linked list, stack, queue.

- A sorted array: search $O(\log n)$, but insert $O(n)$ — everything shifts.
- A linked list: insert $O(1)$ once you are there, but search $O(n)$.

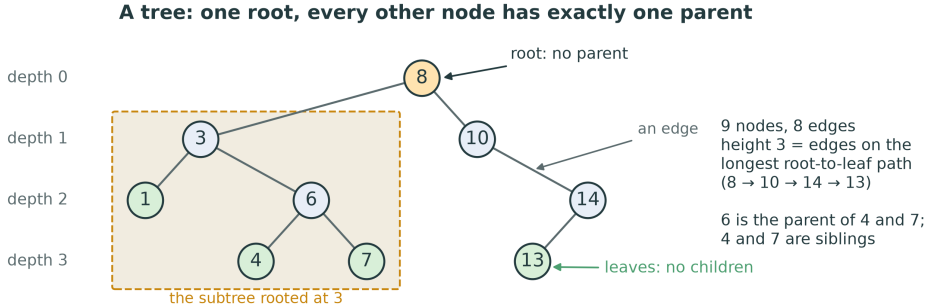
Today: a structure where search **and** insert cost $O(\log n)$ — **while it stays balanced**.

# Today

1. Vocabulary: root, leaf, depth, height, subtree
2. Binary trees, and the `TreeNode`
3. The BST property — and why checking parents is not enough
4. Search, insert, min and max: one path down
5. Delete: three cases, and the in-order successor
6. The four traversals — and what each one is **for**
7. When the tree is a linked list: measured

# The Vocabulary

# A tree, and its words



# Depth and height

- **Depth** of a node: edges from the **root** down to it. The root has depth 0.
- **Height** of a node: edges on the longest path from it **down** to a leaf. A leaf has height 0.
- **Height of the tree** = height of the root. Empty tree: $-1$.

| | depth — العم| | height — الارتفا| |
|---|---|---|
| measured from | the root, down | the leaves, up |
| the root | 0 | the tree's height |
| a leaf | its level | 0 |

# Trees are everywhere

- your **file system**: folders inside folders
- a web page: the HTML **DOM**
- an **expression** — `(3 + 4) * 2` — as your compiler sees it
- a **decision** tree; a tournament bracket; a family tree
- the **recursion tree** of `fib(n)` (Lecture 03)

# Binary Trees

# At most two children

A **binary tree**: every node has at most two children, a **left** and a **right**. In Arabic: شجرة ثنائي.

```
class TreeNode:
    __slots__ = ("value", "left", "right")

    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

Lecture 05's `Node` had one link, `next`. This one has two.

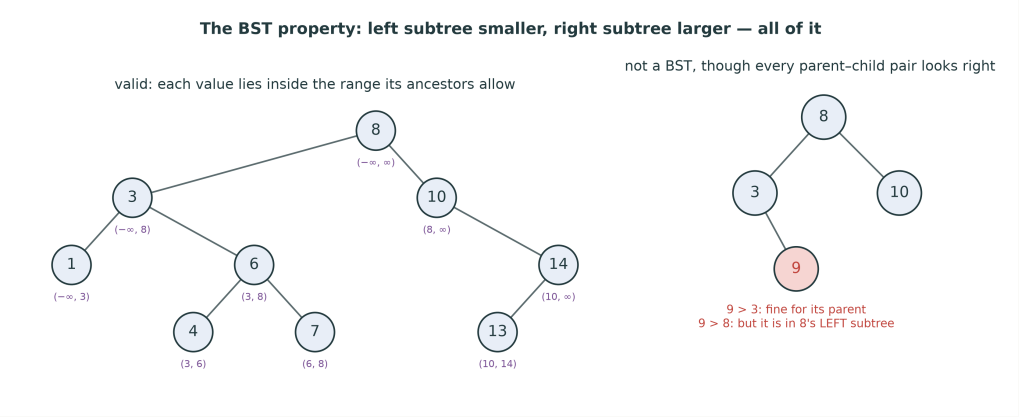
# How tall must a tree be?

- Level d holds **at most** $2^d$ nodes: 1, 2, 4, 8, …
- A tree of height h holds **at most** $2^{h+1} - 1$ nodes.
- So n nodes need height **at least** $\lfloor \log_2 n \rfloor$ — and may have height up to $n - 1$.

| n | shortest possible height | tallest possible height |
|---|---|---|
| 7 | 2 | 6 |
| 1,000 | 9 | 999 |
| 1,000,000 | 19 | 999,999 |

# Binary Search Trees

# The BST property




For **every** node x: everything in x's **left subtree** is smaller than x, and everything in its **right subtree** is larger.

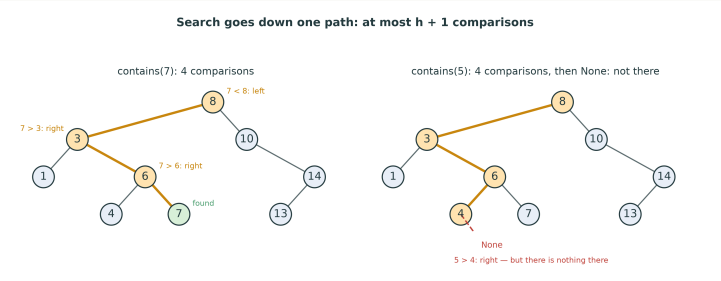
# `is_valid`: carry the range down

The wrong check, at each node: `left.value < node.value < right.value`.

The right check: pass an allowed range **(low, high)** down the recursion.

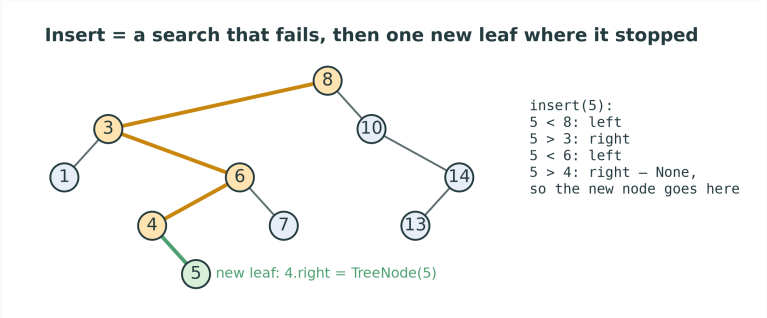
- going **left** from x: the new high is x
- going **right** from x: the new low is x
- the empty tree is valid

# Search: one path down




```
node = self.root
while node is not None:
    if value == node.value:
        return True
    node = node.left if value < node.value else node.right
return False
```

# Insert: a search that fails




1. Empty tree $\rightarrow$ the new node **is** the root.
2. Otherwise walk down as in search. Equal $\rightarrow$ already there: do nothing.
3. When the way on is `None`, hang the new node **there**.

A new value always becomes a **leaf**. Nothing moves: $O(h)$.

# Min and max: no comparisons at all

- **min**: from the root, go **left** until there is no left child.
- **max**: from the root, go **right** until there is no right child.
- Empty tree $\rightarrow$ `ValueError`.

$O(h)$, and not a single comparison of values.

# Delete: Three Cases

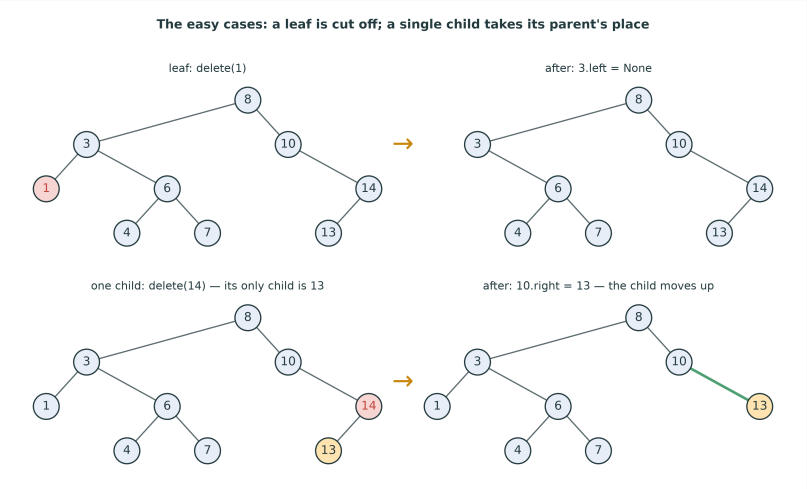
# Find it, then look at its children

Find the node, **remembering its parent**. Then:

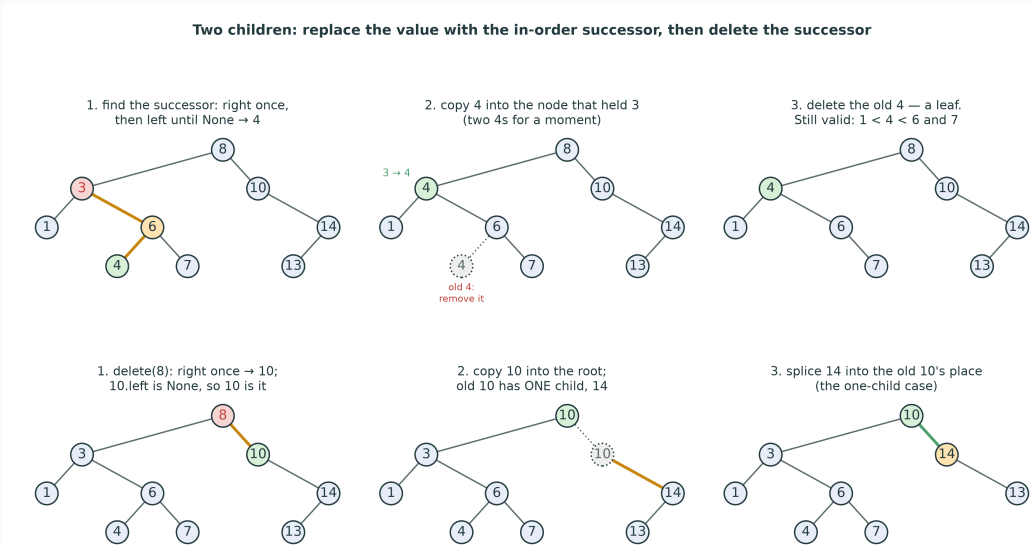
| The node has… | Do this |
|---|---|
| no children (a **leaf**) | the parent's link becomes `None` |
| **one** child | the parent's link points at that child |
| **two** children | copy in the **in-order successor**, then delete the successor |

Deleting a value that is not there does nothing.

# The easy cases



# Two children: the in-order successor



# The three cases collapse into two

1. Find the node and its parent. Not found → return.
2. **Two children?** Walk to the successor (keeping *its* parent), copy its value up — and from now on, delete **the successor node** instead.
3. Now the node has **at most one** child, `child` (possibly `None`):
   - no parent → `self.root = child`
   - node was its parent's left → `parent.left = child`
   - otherwise → `parent.right = child`

# The Four Traversals

# Recursion on trees

A **traversal** visits **every** node once — in Arabic, اجتياز الشجر. For a binary tree, every recursive walk has the same skeleton:

- **base case:** the empty tree — do nothing
- **recursive case:** do something with the node, and walk the left and right subtrees

*Where* you put "do something" gives three different orders.

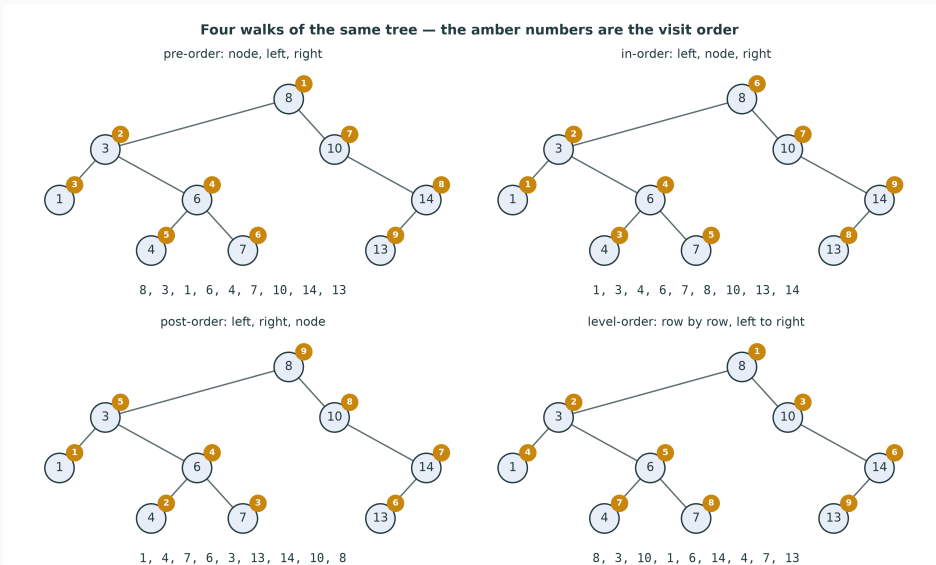
# In-order: the code

```
def in_order(self):
    out = []
    self._in_order(self.root, out)
    return out

def _in_order(self, node, out):
    if node is not None:
        self._in_order(node.left, out)
        out.append(node.value)
        self._in_order(node.right, out)
```

Move `out.append` **up** one line → pre-order. **Down** one line → post-order.

# Four walks of one tree

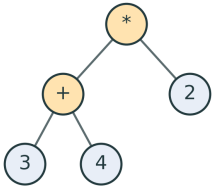


# What each one is FOR

| Walk | Gives you | Used for |
|---|---|---|
| **in-order** | a BST's values **sorted** | listing, range queries, `is_valid` |
| **pre-order** | parents before children | **copying** / saving a tree: re-inserting it **rebuilds** the same shape |
| **post-order** | children before parents | **freeing** a tree; **evaluating** an expression; sizes and heights |
| **level-order** | nearest to the root first | printing by level; shortest distances (BFS, Week 14) |

# Post-order evaluates: expression trees

**An expression tree for (3 + 4) * 2: the shape holds the precedence**




| | | |
|---|---|---|
| `pre-order` | `* + 3 4 2` | prefix (Polish) notation |
| `in-order` | `3 + 4 * 2` | infix — but the brackets are lost! |
| `post-order` | `3 4 + 2 *` | postfix: Lecture 06's stack evaluates it → 14 |

# Level-order needs a queue

**level_order(): the queue holds the nodes found but not yet visited**

| step | children added | queue after the step (front on the left) |
|---|---|---|
| start | | 8 |
| dequeue 8 | enqueue 3, 10 | 3 10 |
| dequeue 3 | enqueue 1, 6 | 10 1 6 |
| dequeue 10 | enqueue 14 | 1 6 14 |
| dequeue 1 | – | 6 14 |
| dequeue 6 | enqueue 4, 7 | 14 4 7 |
| dequeue 14 | enqueue 13 | 4 7 13 |
| dequeue 4 | – | 7 13 |
| dequeue 7 | – | 13 |
| dequeue 13 | – | empty: stop |

visited: 8, 3, 10, 1, 6, 14, 4, 7, 13

1. Enqueue the root.
2. While the queue is not empty: **dequeue** a node, visit it, **enqueue** its children (left, then right).

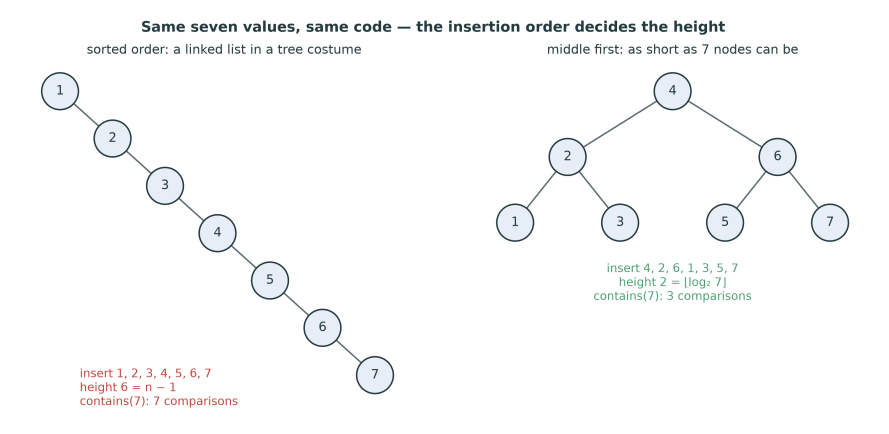
# Height and size: post-order in one line

```
def _height(self, node):
    if node is None:
        return -1
    return 1 + max(self._height(node.left), self._height(node.right))
```

`size` is the same shape: 0 for `None`, else $1+$ left size $+$ right size.

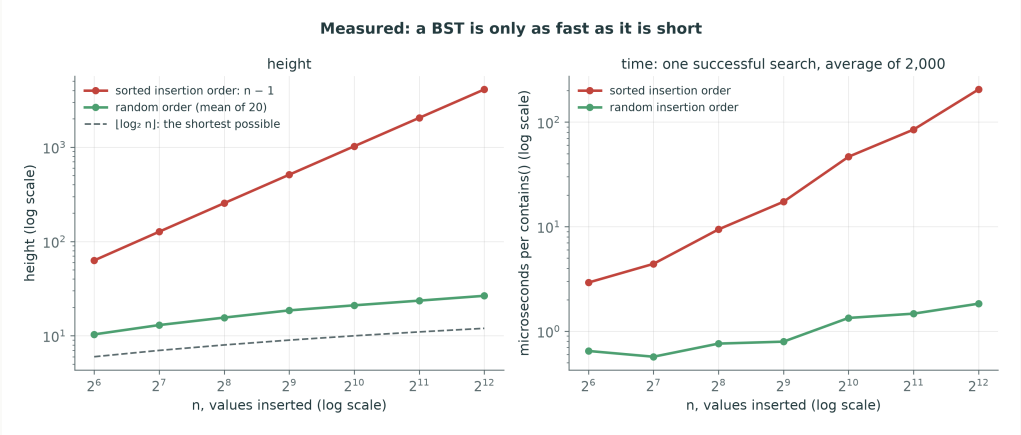
# When the Tree Is a Linked List

# Same values, same code, different height




Insert **sorted** values and each new one is the largest so far: it always goes **right**. Height $n - 1$. Every operation: $O(n)$.

# Measured


Measured: a BST is only as fast as it is short
height
sorted insertion order: n − 1
random order (mean of 20)
⌊log₂ n⌋: the shortest possible
height (log scale)
n, values inserted (log scale)
time: one successful search, average of 2,000
sorted insertion order
random insertion order
microseconds per contains() (log scale)
n, values inserted (log scale)

# The cure: keep it balanced — beyond this course

**Balanced** trees re-shape themselves on every insert and delete, so that h stays $O(\log n)$ whatever the input order:

- **AVL trees** (Adelson-Velsky and Landis, 1962)
- **red-black trees** (Guibas and Sedgewick, 1978)

Both are **beyond the bylaw**: named here, not examined.

# What each operation costs

| Operation | Cost | Short tree | Chain |
|---|---|---|---|
| `contains`, `insert`, `delete` | $O(h)$ | $O(\log n)$ | $O(n)$ |
| `min`, `max` | $O(h)$ | $O(\log n)$ | $O(n)$ |
| `height`, `size`, `is_valid` | $O(n)$ | | |
| the four traversals | $O(n)$ | | |
| extra space, recursive walks | $O(h)$ | $O(\log n)$ | $O(n)$ |
| extra space, `level_order` | $O(w)$ | up to $O(n)$ | $O(1)$ |

# This Week

# Exercises: `dsa/tree.py`

| Method | Target | The trap |
|---|---|---|
| `insert` | $O(h)$ | stop **at the parent**; ignore duplicates |
| `contains`, `min`, `max` | $O(h)$ | a loop; `ValueError` on empty |
| `delete` | $O(h)$ | keep the parent; the **root**; successor = right child |
| `height`, `size` | $O(n)$ | empty tree is $-1$ and 0 |
| `is_valid` | $O(n)$ | carry **(low, high)** down, not just the parent |
| the four traversals | $O(n)$ | return a **new list**; level-order uses **your** queue |

```
pytest tests/test_tree.py -v
```

# Homework 11 — before Lecture 12

1. **Implement** `dsa/tree.py` until all 26 tests pass.
2. **Draw** the BST built by inserting 50, 30, 70, 20, 40, 60, 80, 35, 45, 65, then delete 20, 30 and 50 — the tree after each delete.
3. **Walk** your tree of item 2 (before the deletes) in all four orders, by hand. Check with your code.
4. **Measure** the height of a BST built from `range(n)` and from a shuffled `range(n)`, for n up to 900, in `notebooks/11-trees.ipynb`. Why stop at 900?

# Practise online

- **Week 11 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 11

# Summary

# Seven things to keep

1. A tree: one root; every other node has **one parent**; n nodes, $n - 1$ edges.
2. **Height** counts edges down to the deepest leaf; empty tree $-1$, leaf 0.
3. BST: **left subtree smaller, right subtree larger** — for the whole subtree.
4. Search, insert, min, max: **one path down**, $O(h)$.
5. Delete with two children: copy the **in-order successor**, delete it instead.
6. In-order is **sorted**; pre-order **rebuilds**; post-order **frees and evaluates**; level-order needs a **queue**.
7. Sorted input gives height $n - 1$: $O(n)$. Balanced trees fix it — beyond this course.

# Next

**Week 12 — Heaps and priority queues.** A tree with no links at all: a **complete** binary tree stored in an `Array`, where the children of slot i are slots 2i + 1 and 2i + 2.